

Thiết kế hàm mất mát (Loss Function Design)

PHỤ THUỘC B/04-nen-tang-hoc-tu-du-lieu (ERM và *inductive bias* — loss là một trong bốn chỗ cài giả định); B/05-thiet-ke-bai-toan (đầu ra và tín hiệu giám sát); A/01-toan-toi-thieu/03-xac-suat-va-thong-tin (KL, entropy, ước lượng độ hợp lý cực đại).

TÓM TẮT

Chương này nói về khoảng cách giữa ba thứ hay bị gọi chung là “mục tiêu”: điều bạn thật sự muốn, phép đo bạn báo cáo, và hàm khả vi bạn thực sự tối ưu. Nó cho một nguyên lý duy nhất sinh ra hầu hết công thức loss: mọi loss là một phát biểu về phân phối. Các họ loss còn lại ra đời vì phát biểu mặc định đó không khớp với một thực tế nào đó. Đọc xong, bạn chọn được loss bằng cách trả lời một câu hỏi về dữ liệu thay vì tra bảng công thức. Bạn chẩn đoán được “loss giảm mà kết quả không tốt lên”. Và bạn nhìn ra chỗ một loss bị *phẳng*, nơi mô hình mù hoàn toàn, trước khi nó kịp gây hại. Nếu chỉ nhớ một điều: loss không phải công thức mà là một phát biểu về nhiễu, về mẫu nào đáng được chú ý, và về loại sai lầm nào đắt hơn; nếu bạn không nói được phát biểu đó thành một câu tiếng Việt thì bạn mới sao chép loss chứ chưa chọn nó.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình và tham số** — “mô hình” ở đây là một hàm số có hàng triệu con số điều chỉnh được gọi là tham số (*parameter*, hay trọng số); huấn luyện nghĩa là tìm bộ con số đó. Siêu tham số (*hyperparameter*) thì ngược lại: bạn đặt bằng tay trước khi huấn luyện và thuật toán không tự sửa.
- **gradient và hạ gradient** — gradient là vectơ đạo hàm của một hàm theo mọi tham số, tức hướng làm hàm đó tăng nhanh nhất. Huấn luyện là lặp đi lặp lại việc đi ngược hướng gradient một bước nhỏ. Hệ quả quan trọng cho cả chương: nơi nào hàm mất mát *phẳng*, gradient bằng 0 và mô hình không nhận được tín hiệu học nào từ chỗ đó.
- **logit và softmax** — logit là điểm số thô mà mạng xuất ra cho mỗi lớp, có thể âm hoặc lớn tùy ý. Softmax là phép biến đổi exp rồi chuẩn hoá, đưa các logit về một bộ số dương cộng lại bằng 1 để đọc được như xác suất.
- **nhãn one-hot** — cách mã hoá nhãn đúng thành một vectơ toàn số 0 trừ một số 1 ở vị trí lớp đúng. Nó phát biểu “lớp này đúng tuyệt đối, mọi lớp khác sai tuyệt đối” — một phát biểu mạnh hơn thực tế, và vài kỹ thuật trong chương ra đời chính vì điều đó.
- **độ hợp lý (likelihood)** — với một mô hình xác suất và một tập dữ liệu, độ hợp lý là xác suất mà mô hình gán cho chính dữ liệu đã quan sát. “Cực đại hoá độ hợp lý” nghĩa là chọn tham số làm dữ liệu thật trở nên khả dĩ nhất có thể.
- **metric so với hàm mất mát** — metric là con số bạn *báo cáo* (accuracy, mAP, IoU) — thường không khả vi nên không dùng để học được. Hàm mất mát là con số bạn *tối ưu*. Toàn bộ chương này nói về khoảng cách giữa hai thứ đó.
- **IoU và Dice** — IoU (*intersection over union*) là diện tích phần giao chia cho diện tích phần hợp của hai vùng — thước đo chuẩn cho hộp và mặt nạ. Dice là biến thể lấy hai lần phần giao chia cho tổng hai diện tích; cùng tinh thần, khác mẫu số.
- **hiệu chỉnh xác suất (calibration)** — mức độ mà xác suất mô hình xuất ra khớp với tần suất đúng thật sự: nếu mô hình nói “chắc 80%” trên một nghìn mẫu thì khoảng tám trăm mẫu phải đúng. Một mô hình phân loại chính xác vẫn có thể hiệu chỉnh rất tệ, và điều đó quan trọng khi có ngưỡng ra quyết định ở hạ nguồn.

CÂU HỎI MỞ ĐẦU

- Q1. Hai mô hình có cùng giá trị hàm mất mát trên cùng một tập, vì sao một cái dùng được để ra quyết định có ngưỡng còn cái kia thì không?
- Q2. Nếu bạn chưa bao giờ ngồi chọn giả định về phân phối của nhiễu, vì sao chỉ cần gỡ MSELoss là bạn đã chọn xong rồi?
- Q3. Vì sao một mô hình phân vùng đạt độ chính xác pixel 99% lại có thể chưa từng đoán đúng một pixel tiền cảnh nào?
- Q4. Hai hộp dự đoán cùng trượt khỏi hộp thật, một cái trượt sát và một cái trượt rất xa. Vì sao thước đo chuẩn của bài toán không phân biệt được chúng, và mô hình học được gì từ chỗ đó?
- Q5. Vì sao cổ tình bôi bẩn nhãn đúng — nói rằng lớp đúng chỉ đúng 90% — lại làm mô hình tốt lên ở một khía cạnh và tệ đi ở khía cạnh khác?

1.1 Đặt vấn đề

Mô hình tối ưu chính xác cái bạn viết ra, không phải cái bạn nghĩ trong đầu. Câu này nghe hiển nhiên tới mức nhàm, nhưng nó là nguồn của phần lớn những buổi chiều mà loss giảm đều đặn còn hệ thống thì tệ đi. Gốc rễ nằm ở chỗ có *ba* thứ khác nhau thường bị gọi chung là “mục tiêu”, và chúng nằm trên một chuỗi xấp xỉ mà mỗi bước đều rò rỉ:

- **Mục tiêu thực tế** — giảm số vụ va chạm của con robot. Hầu như không bao giờ khả vi, thường còn không đo được trong lúc huấn luyện: nó là biến cố hiếm, quan sát được sau nhiều tháng, và phụ thuộc cả những khối không phải mạng nơ-ron.
- **Metric đánh giá** — mAP trên tập test. Đo được ngay, nhưng là hàm bậc thang theo tham số: thay trọng số một lượng vô cùng nhỏ thì nó hoặc đứng yên hoặc nhảy một bậc, nên *gradient* của nó bằng 0 gần như mọi nơi và vô dụng cho việc học.
- **Hàm mất mát khả vi** — một tổ hợp *focal* cộng GloU cộng vài số hạng nữa. Trơn, có gradient khác 0 ở mọi chỗ ta cần, và ta *hy vọng* đẩy nó xuống thì kéo metric lên.

Chữ “hy vọng” ở dòng cuối là một phát biểu toán học có tên: tính **nhất quán** (*consistency*) của **hàm thay thế** (*surrogate loss*) — điều kiện để nghiệm tối ưu của hàm thay thế trùng với nghiệm tối ưu của mục tiêu gốc. Khi điều kiện đó không thành lập, bạn đang tối ưu sai thứ một cách có hệ thống, và không có lượng dữ liệu nào cứu được. Đây là chỗ mà chương này nổi thẳng về chương 4: hàm mất mát là một trong bốn chỗ cài *inductive bias*, và cũng như ba chỗ kia, giả định cài vào có thể sai.

→ **tiền đề** B/04 — bốn nguồn của inductive bias — kiến trúc, hàm mất mát, dữ liệu, thuật toán tối ưu — được liệt kê ở đó; chương này khai triển nguồn thứ hai.

Câu hỏi dẫn dắt do đó không phải “có những loss nào” mà là: *làm sao viết ra đúng thứ bạn muốn, khi thứ bạn muốn thường không khả vi?*

TRỰC GIÁC

Gradient chỉ nhìn thấy *độ dốc cục bộ* của loss, nên chỗ nào loss phẳng thì chỗ đó mô hình mù hoàn toàn. Hãy đọc mọi hàm mất mát như một bản đồ độ dốc: nó quyết định mẫu nào còn “phát ra tín hiệu” và mẫu nào đã tàng hình. **Hinge loss** cố tình làm phẳng vùng đã vượt lề, nên mẫu phân đúng biến mất khỏi tầm nhìn; **entropy chéo** (*cross-entropy*) không bao giờ phẳng, nên một mẫu đã đúng với xác suất 0,99 vẫn tiếp tục kéo mô hình. Focal loss là hành động ngược lại: *cố ý* bơm vùng phẳng vào chỗ dễ để tín hiệu của mẫu khó nổi lên.

Nhìn theo bản đồ độ dốc thì ba công thức trông rất khác nhau kia trở thành ba lựa chọn của cùng một câu hỏi: “tôi muốn ai còn nói được?”

1.2 Loss là một phát biểu xác suất (Loss as a Probabilistic Statement)

A. Một nguyên lý sinh ra hầu hết công thức

Có hàng chục công thức loss trong các thư viện, và học thuộc chúng là cách tệ nhất để dùng chúng. Có một nguyên lý sinh ra gần hết: **cực đại hoá độ hợp lý** (*maximum likelihood estimation*). Giả sử bạn mô hình hoá dữ liệu bằng một phân phối có điều kiện $p_\theta(y | x)$; khi đó tối thiểu hoá $-\sum_i \log p_\theta(y_i | x_i)$ chính là ước lượng độ hợp lý cực đại, và mọi thứ còn lại chỉ là chọn họ phân phối:

Gaussian $\Rightarrow \frac{1}{2}\|y - \hat{y}\|^2$ (MSE), Laplace $\Rightarrow |y - \hat{y}|$ (MAE), Categorical $\Rightarrow -\log \hat{p}_y$ (CE).

Ý nghĩa của ba dòng trên, nói gọn: *bạn không cần nhớ công thức nào cả, chỉ cần trả lời một câu — tôi giả định phần dư phân bố thế nào?* Nếu bạn tin sai số là Gaussian thì bạn đã chọn MSE dù không biết mình chọn; nếu dữ liệu có ngoại lai nặng đuôi thì giả định Gaussian sai, và MSE để một mẫu hỏng kéo cả mô hình vì nó phạt bậc hai [1].

B. Chính quy hoá là prior, không phải mẹo

Nguyên lý này mở rộng tự nhiên. Thay *MLE* bằng **cực đại hoá hậu nghiệm** (*maximum a posteriori*) thì thêm một số hạng $-\log p(\theta)$: prior Gaussian cho phạt L2, prior Laplace cho phạt L1 và tính thừa đi kèm [2]. Điều này biến câu “thêm *weight decay* cho đỡ *overfitting*” — một câu vô nghĩa về mặt nội dung — thành một phát biểu kiểm chứng được: *tôi tin rằng trước khi nhìn dữ liệu, trọng số nhỏ khả dĩ hơn trọng số lớn*. Và như mọi niềm tin, nó có thể sai: với mạng có **chuẩn hoá** (*normalization*), phạt L2 lên trọng số mất phần lớn nghĩa hình học vì thang của trọng số bị chuẩn hoá triệt tiêu ngay sau đó.

Một hệ quả ít được nói: nếu loss là log-likelihood thì đầu ra của mô hình là *tham số của một phân phối*, không phải một con số. Đi theo hướng đó ta được *heteroscedastic loss* — mạng xuất ra cả $\hat{y}$ lẫn $\log \sigma^2$ và tối thiểu hoá $\frac{(y - \hat{y})^2}{2\sigma^2} + \frac{1}{2} \log \sigma^2$. Số hạng thứ nhất cho phép mô hình giảm phạt ở mẫu nó tự nhận là khó; số hạng thứ hai ngăn nó tuyên bố mọi thứ đều khó. Kết quả là mô hình học luôn độ bất định của chính mình, và độ bất định đó dùng được ở hạ nguồn [3]. Đó đúng là thứ một bộ lọc hợp nhất cảm biến cần, và là chỗ nối tự nhiên nhất giữa chương này với nền tảng ước lượng trạng thái bạn đã có.

C. Từ “chọn công thức” tới “phát biểu giả định”

Mạch 1 — hồi quy, đọc theo giả định chứ không theo tên

1 Vấn đề

Giải pháp

Hệ quả

có quá nhiều công thức loss để nhớ, và không có tiêu chí chọn.

coi mọi loss là $-\log p_\theta(y | x)$ và chỉ chọn họ phân phối.

câu hỏi “dùng MSE hay MAE” trở thành “phần dư của tôi có đuôi nặng không”, một câu trả lời được bằng cách vẽ biểu đồ phần dư thay vì bằng cách thử cả hai rồi chọn con số đẹp hơn.

2 Vấn đề

giả định Gaussian sai khi có ngoại lai, nhưng MAE có gradient hằng nên hội tụ chậm quanh nghiệm.

- Giải pháp** **hàm Huber** (*Huber loss*) — bậc hai trong bán kính δ , tuyến tính ngoài bán kính đó.
- Giả định** ngoại lai là hiếm và nằm xa; δ tách được hai chế độ.
- Giá phải trả** thêm một siêu tham số, và nếu δ đặt sai thì bạn được cái tệ nhất của cả hai. Người làm SLAM nhận ra ngay đây chính là hàm robust quen thuộc trong **hiệu chỉnh chùm tia** (*bundle adjustment*), và trực giác từ đó chuyển sang nguyên vẹn.
- 3 Vấn đề** đôi khi ta không cần một điểm mà cần một khoảng.
- Giải pháp** **quantile loss** — phạt bất đối xứng để mô hình học phân vị thứ τ thay vì kỳ vọng; ba đầu ra ở $\tau = 0,1; 0,5; 0,9$ cho một khoảng dự đoán mà không cần giả định Gaussian nào.
- Vấn đề mới** nếu loss là giả định phân phối thì *prior cũng vậy* → chính quy hoá thôi là mẹo và trở thành một phát biểu có thể sai, có thể kiểm chứng, và có thể gỡ ra khi sai.

I 1.3 Phân loại: chọn loss là chọn hành vi (Classification: Choosing Behaviour)

A. Hai bản đồ độ dốc trên cùng một bài toán

Bảng 1.1. Hinge và cross-entropy: cùng một bài toán, hai hành vi hoàn toàn khác nhau.

Tiêu chí	Hinge (multiclass SVM)	Cross-entropy (softmax)
Cơ chế	Phạt cho tới khi vượt lề Δ , sau đó gradient tắt hẳn	Phạt – log xác suất đúng, không bao giờ tắt
Giả định ngầm	Chỉ cần phân đúng với một lề an toàn là đủ	Ta cần cả xác suất, không chỉ nhãn
Đầu ra	Điểm số, không diễn giải được như xác suất	Xác suất dùng được ở hạ nguồn
Ngoại lai đã phân đúng	Bị bỏ qua hoàn toàn	Vẫn kéo mô hình
Mạnh khi nào	Cần biên quyết định ổn định, ít quan tâm độ tin cậy	Cần ngưỡng hoá, hợp nhất, hoặc chưng cất về sau
Hổng khi nào	Khi quyết định hạ nguồn cần một xác suất có nghĩa	Nhãn nhiều: mô hình bị kéo vô hạn về phía nhãn sai

Bài học tổng quát vượt ra ngoài hai công thức: *chọn loss là chọn hành vi*. Đặc tính “không bao giờ tắt” của cross-entropy là ưu điểm khi ta cần xác suất được hiệu chỉnh, và là nhược điểm chí mạng khi trong tập có 5 % nhãn sai — mô hình sẽ dành phần lớn sức chứa cuối buổi huấn luyện để ghi nhớ đúng những nhãn sai đó [4].

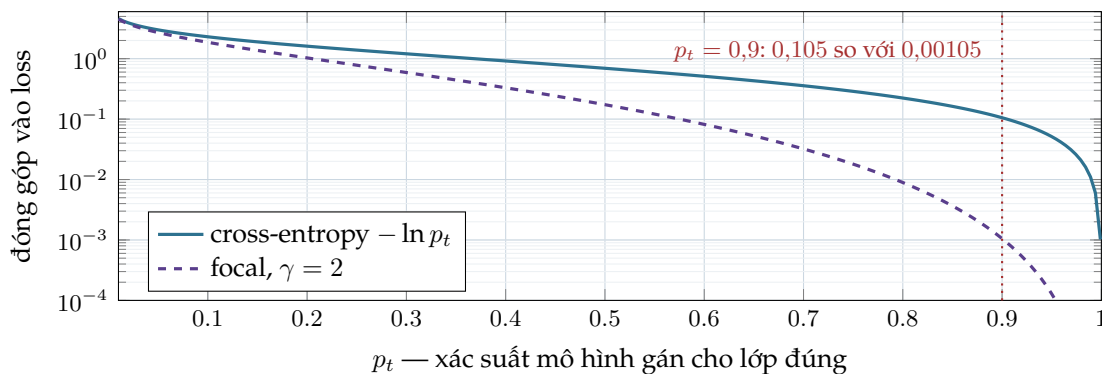
B. Ba cách định hình lại sự chú ý

Cross-entropy cộng đều đóng góp của mọi mẫu. Trong một bài phát hiện vật thể một giai đoạn, số *anchor* nên có thể gấp mười nghìn lần số anchor có vật; dù mỗi anchor nên chỉ đóng góp một lượng nhỏ, tổng của chúng nhấn chìm tín hiệu. Ba sửa chữa phổ biến tấn công ba chỗ khác nhau, và nhằm lẫn giữa chúng là lỗi rất hay gặp.

I MẠCH 2 — ba cơ chế khác nhau cho ba triệu chứng khác nhau

- 1 Vấn đề** mẫu dễ áp đảo về số lượng.
Giải pháp *focal loss*: nhân CE với $(1 - p_t)^\gamma$ [5].
Giả định đã dễ thì đã học xong, không cần học thêm.
Giá phải trả một siêu tham số γ , và nếu nhân nhiều thì focal *khuếch đại* nhiều, vì một mẫu có nhãn sai luôn trông y hệt một mẫu khó.
- 2 Vấn đề** lớp hiếm ít mẫu — đây là chuyện khác hẳn “mẫu dễ nhiều”.
Giải pháp *class-balanced loss* — cân theo số mẫu hiệu dụng của từng lớp.
Giả định nguyên nhân là tần suất, không phải độ khó hình ảnh.
Hệ quả trên một tập đuôi dài mà mọi lớp đều dễ về mặt thị giác, focal không giúp gì còn class-balanced thì giúp; trên một tập cân bằng nhưng có vài mẫu cực khó thì ngược lại.
- 3 Vấn đề** mô hình quá tự tin, xác suất không dùng được.
Giải pháp *làm mượt nhãn* (*label smoothing*): thay nhãn one-hot bằng $(1 - \varepsilon)$ cho lớp đúng và $\varepsilon/(C - 1)$ cho phần còn lại [6].
Hệ quả nó đặt trần hữu hạn cho logit đúng nên chấm dứt cuộc đua logit vô hạn.
Giá phải trả nó cũng làm co cụm các lớp trong không gian đặc trưng, và điều đó *làm hại* nếu bạn định dùng chính đặc trưng đó cho truy hồi hoặc chứng cất.

Hình 1.1 vẽ ra điều mà hộp trực giác vừa nói bằng lời: nó cho thấy chính xác vùng nào của trực xác suất bị focal loss “tắt tiếng”.



Hình 1.1. Vì sao focal loss “tắt tiếng” của mẫu dễ. Ở $p_t = 0,9$, hệ số $(1 - p_t)^2 = 0,01$ chia đóng góp cho một trăm lần; ở $p_t = 0,1$ hệ số là 0,81, gần như không đổi. Tỷ lệ chú ý giữa mẫu khó và mẫu dễ do đó bị nhân lên khoảng tám mươi lần. Đó cũng là lý do một nhãn sai được khuếch đại đúng chừng ấy: với mô hình, nhãn sai trông y hệt một mẫu cực khó.

C. Chiều của KL quyết định mô hình sinh ra thứ gì

Một trực thứ tư ít được nêu nhưng quyết định hành vi của cả họ mô hình sinh: *đo khoảng cách giữa hai phân phối theo chiều nào*. $KL(p \parallel q)$ (*forward*, tức cross-entropy quen thuộc) phạt rất nặng khi q gán xác suất gần 0 vào chỗ p có khối lượng, nên nghiệm tối ưu *phủ hết các mode* và bôi trơn qua mọi khả năng. $KL(q \parallel p)$ (*reverse*) chỉ phạt khi q đặt khối lượng vào chỗ p không có, nên nghiệm tối ưu *bám lấy một mode* và bỏ qua phần còn lại. Hệ quả rất cụ thể: hướng phủ mode sinh ra thứ trung bình mờ nhạt, hướng tìm mode sinh ra thứ sắc nét nhưng thiếu đa dạng. Chương 17 dùng lại đúng trực này để giải thích vì sao VAE mờ còn GAN thì sắc mà hay sập mode.

↔ **tiền đề** C/17 — bốn họ mô hình sinh khác nhau ở chỗ chúng chọn chiều nào của KL, hoặc thay KL bằng một định nghĩa “giống” khác hẳn.

1.4 Khi đầu ra có cấu trúc (Structured Outputs)

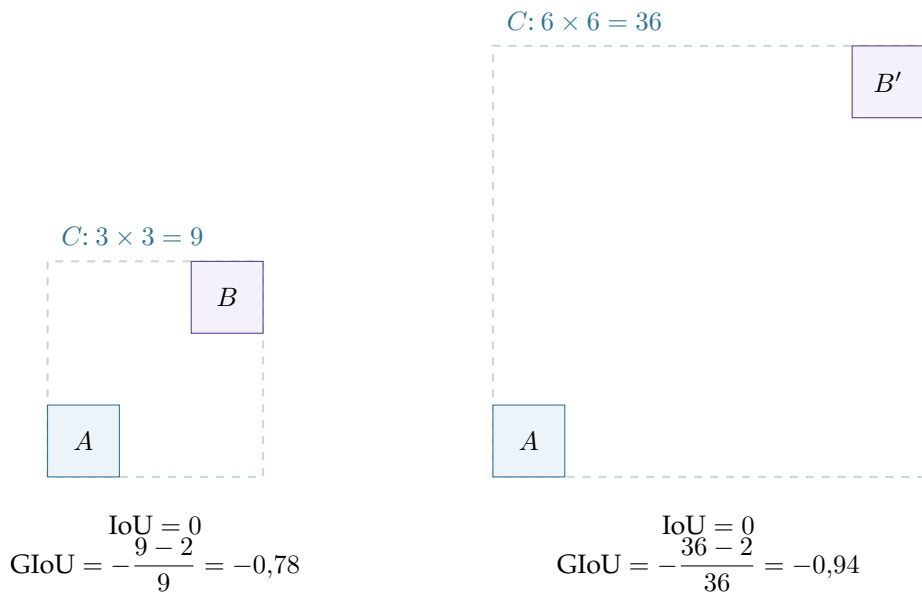
Nếu đầu ra là một mặt nạ, một hộp, một chuỗi hay một tập hợp thì trung bình theo từng phần tử là xấp xỉ tồi của thứ ta thật sự quan tâm. Nguyên tắc chung: *loss phải tôn trọng cấu trúc của đầu ra*, và cách làm điển hình là tìm một xấp xỉ khả vi cho chính metric của bài toán.

A. Phân vùng: vì sao Dice không bị đánh lừa

Với **phân vùng** (*segmentation*), CE theo từng pixel bị chi phối bởi lớp nền. Lấy ví dụ nhỏ nhất tính tay được: ảnh 100×100 với đúng 100 pixel tiền cảnh, tức 1 %. Một mô hình đoán “toàn bộ là nền” đạt độ chính xác pixel 99 % và CE thấp một cách đáng khen, trong khi hệ số **Dice** (*Dice coefficient*) $= 2|A \cap B|/(|A| + |B|) = 2 \cdot 0/(0 + 100) = 0$ — bằng không tuyệt đối [7]. Dice không bị đánh lừa vì nó chỉ đếm phần giao trên tổng diện tích hai mặt nạ, và pixel nền đúng không được cộng điểm. Cái giá là gradient của Dice phụ thuộc *toàn ảnh* chứ không tách được theo từng pixel, nên nó ổn hơn khi *batch* nhỏ và có mẫu số tiến về 0 khi ảnh không chứa tiền cảnh nào. Trong thực hành người ta hầu như luôn dùng tổ hợp CE cộng Dice, còn **Tversky** (*Tversky loss*) là bản tổng quát cho phép đặt giá khác nhau cho dương tính giả và âm tính giả — tức là chọn mặt nạ “béo” hay “gầy” ngay từ công thức, trước khi nhìn kết quả.

B. Hộp: lỗ hổng gradient của IoU và cách GIoU và nó

Với **hộp giới hạn** (*bounding box*), L1 trên bốn tọa độ không phải thứ ta đo, vì metric là IoU. Nhưng IoU có một lỗ hổng chí mạng và nó tính tay được, như Hình 1.2 cho thấy.



Hình 1.2. Hai dự đoán rời hoàn toàn khỏi hộp thật A . IoU bằng 0 trong cả hai trường hợp, nên loss phẳng và mô hình không biết B tốt hơn B' . GIoU trừ đi phần thừa của hộp bao nhỏ nhất C và nhờ vậy phân biệt được hai tình huống, cho gradient chỉ đúng hướng.

Công thức là $\text{GIoU} = \text{IoU} - (|C| - |A \cup B|)/|C|$ [8], và ý nghĩa của số hạng thứ hai bằng lời: *phạt theo mức độ “lãng phí” của hộp bao nhỏ nhất chứa cả hai hộp* — càng xa nhau thì hộp bao càng rộng, càng bị phạt nặng. Đây là ví dụ mẫu mực cho cả chương: người ta không phát minh GIoU vì nó đẹp, mà vì họ đi tìm đúng chỗ loss bị phẳng.

C. Chuỗi và tập hợp

Cùng một nguyên tắc sinh ra hai họ nữa. CTC xử lý chuỗi không cần chỉnh bằng cách cộng xác suất trên mọi cách căn chỉnh hợp lệ, biến một bài toán không khả vi vì thiếu nhãn từng khung thành một bài toán khả vi. Và **ghép cặp Hungary** (*Hungarian matching*) xử lý đầu ra dạng tập hợp: DETR bỏ hẳn NMS bằng cách ghép một-một tối ưu giữa dự đoán và nhãn *trước* khi tính loss, khiến loss trở nên bất biến với thứ tự [9]. Cả hai minh hoạ cùng một câu. Khi metric của bài toán không có xấp xỉ khả vi sẵn, bạn phải tự thiết kế một cái. Việc thiết kế nằm ở chỗ gỡ đúng điểm không khả vi, không phải ở chỗ làm công thức phức tạp hơn.

1.5 Ba nguồn giám sát ngoài nhãn lớp (Supervision Beyond Class Labels)

Ba họ loss còn lại chia nhau một ý tưởng: tín hiệu giám sát có thể đến từ quan hệ giữa các mẫu, từ vật lý, hoặc từ một mô hình khác.

- **Từ quan hệ giữa các mẫu — *metric learning*.** Thay vì hỏi “ảnh này thuộc lớp nào”, ta hỏi “hai ảnh này có giống nhau không”, và tối ưu để cặp dương gần hơn cặp âm một lẽ nào đó [10], [11]. Hai chi tiết quyết định thành bại, và cả hai đều là chỗ hay hỏng:
 - **Mẫu âm** — nếu toàn mẫu âm dễ thì loss về 0 mà biểu diễn chẳng học được gì; nhưng khai thác mẫu âm khó quá tay thì bạn chủ yếu đang huấn luyện trên các dương giả bị gán nhầm là âm.
 - **Nhiệt độ (*temperature*)** — nó điều khiển độ tập trung của phân phối tương tự; nhiệt độ thấp biến *InfoNCE* thành “chỉ quan tâm mẫu âm khó nhất”, tức là đưa cả loss về sát chế độ hỏng vừa nói.

Đây là nền của toàn bộ tự giám sát hiện đại.

↔ **tiền đề C/18** — mọi phương pháp tự giám sát hiện đại đều là một lựa chọn cụ thể của cặp “mẫu dương là gì” và “mẫu âm lấy từ đâu”.

- **Từ vật lý — ràng buộc hình học nhúng vào mục tiêu.** Thị giác máy tính có sẵn một kho tri thức không cần học lại: **sai số tái chiếu** (*reprojection error*), **nhất quán trắc quang** (*photometric consistency*) giữa hai khung nhìn, prior trơn cho trường độ sâu. Đưa chúng vào loss là cách nói “đừng học lại hình học, tôi biết rồi”,

↔ **cùng nguyên lý với C/16** — sai số tái chiếu trong hình học đa khung nhìn và mất mát tự giám sát cho độ sâu là cùng một hàm, chỉ khác chỗ ai giữ biến nào cố định.

và đó chính là cây cầu giữa SLAM cổ điển với học sâu — lý do các phương pháp ước lượng độ sâu tự giám sát huấn luyện được mà không cần một nhãn độ sâu nào. Giả định ngàm — bề mặt Lambert, không có vật chuyển động, phơi sáng không đổi — chính là danh sách các chế độ hỏng của chúng.

- **Từ một mô hình khác. Mất mát tri giác (*perceptual loss*)** so sánh đặc trưng của một mạng đã huấn luyện thay vì so sánh pixel [12], [13]; nó chứa một hiện tượng rất cụ thể. Khi nhiều đáp án đều đúng, loss trung bình theo pixel kéo đầu ra về *trung bình của các đáp án* — mà trung bình của nhiều ảnh sắc nét là một ảnh mờ. Cùng họ, **style loss** qua ma trận Gram cho thấy thống kê bậc hai của đặc trưng mang thông tin kết cấu còn bậc một mang thông tin bố cục [14]. Và **chưng cất tri thức** (*knowledge distillation*) [15] khai thác đúng chỗ nhãn one-hot lãng phí: phân phối mềm của *teacher* nói rằng con mèo này hơi giống con cáo, một thông tin về cấu trúc lớp mà nhãn cứng vứt bỏ hoàn toàn.

1.6 Cân bằng nhiều thành phần (Balancing Multi-Term Losses)

Khi một loss có bốn số hạng, siêu tham số khó nhất của cả hệ thống không nằm trong paper mà nằm trong file cấu hình. Ba triệu chứng cần nhận ra:

- **Thang đo lệch nhau** — một số hạng cỡ 10^{-3} và một số hạng cỡ 10^1 , nên số hạng nhỏ chỉ là trang trí. Kiểm tra bằng cách log riêng từng thành phần và chuẩn của gradient mà nó sinh ra, vì một loss nhỏ vẫn có thể có gradient lớn.
- **Xung đột gradient** — hai số hạng đẩy trọng số về hai hướng có tích vô hướng âm, nên tiến bộ ở nhánh này là thoái bộ ở nhánh kia. Đây là bản chất của học đa nhiệm chứ không phải lỗi cài đặt, nên đừng đi tìm bug.
- **Trọng số không chuyển được** — bộ trọng số tinh chỉnh trên một tập dữ liệu thường vô nghĩa trên tập khác, vì thang của từng thành phần phụ thuộc phân bố dữ liệu.

↪ **ràng buộc bởi** E/24 — hai bộ trọng số chỉ so sánh được khi mọi thứ khác trong giao thức thí nghiệm được giữ cố định — nếu không, bạn đang đo nhiễu.

Có cách đặt trọng số có nguyên lý thay vì dò tay: cho mô hình tự học một σ_k cho mỗi nhánh, rồi cân bằng theo $1/\sigma_k^2$ [3]. Nhưng hãy trung thực rằng đây là kinh nghiệm thực hành chứ không phải định lý. Trong nhiều hệ thống thật, một vòng tìm kiếm ngẫu nhiên trên hai ba trọng số vẫn thắng nó.

Hai khung liên quan đầy ý tưởng đi xa hơn:

- **DRO** — tối thiểu hoá loss của *nhóm tệ nhất* thay vì loss trung bình [16]. Đây là thứ bạn muốn khi trung bình đẹp che giấu một nhóm hỏng hoàn toàn.
- **Cost-sensitive learning** — thôi giả vờ rằng dương tính giả và âm tính giả đắt như nhau. Nếu bỏ sót một người đi bộ đất gấp nghìn lần một cảnh báo thừa, con số đó phải nằm trong loss chứ không phải trong một cuộc họp.

1.7 Thực hành

Repo và tài nguyên. [qubvel-org/segmentation_models.pytorch](https://github.com/qubvel-org/segmentation_models.pytorch) cho Dice, Tversky và focal đã cài sẵn; [KevinMusgrave/pytorch-metric-learning](https://github.com/KevinMusgrave/pytorch-metric-learning) cho toàn bộ họ *contrastive* và các chiến lược khai thác mẫu âm; [lightly-ai/lightly](https://github.com/lightly-ai/lightly) cho InfoNCE; [open-mmlab/mmdetection](https://github.com/open-mmlab/mmdetection) cho GIoU/DIoU/CIoU và Hungarian matching. Với loss nhiều thành phần, hãy đọc code chứ đừng đọc paper: trọng số thật luôn nằm trong config. Danh sách này chốt tại tháng 9/2026 và tên mô hình dẫn đầu sẽ cũ trong 6–12 tháng, trong khi các repo hạ tầng kể trên bền hơn nhiều.

Câu hỏi kiểm tra hiểu. Vì sao một mô hình đạt hinge loss bằng 0 vẫn có thể hiệu chỉnh xác suất rất tệ? Focal loss và class-balanced loss chứa cùng một triệu chứng bằng hai cơ chế khác nhau — khác ở đâu, và tập dữ liệu nào làm lộ ra sự khác biệt? Trong một loss bốn thành phần, làm sao biết thành phần nào đang chi phối gradient?

Mini project. Trên CIFAR-10-LT, huấn luyện cùng một *backbone* với ba cấu hình: CE, CE cộng label smoothing, và focal. Báo cáo ba con số cho mỗi cấu hình — độ chính xác tổng, độ chính xác nhóm đuôi, và ECE. Ba con số kể ba câu chuyện khác nhau, và chỗ chúng mâu thuẫn nhau mới là chỗ đáng viết ra.

1.8 Giới hạn và trường hợp hỏng

Giới hạn lớn nhất là giới hạn khái niệm: loss chỉ là hàm thay thế, và không có gì bảo đảm nó nhất quán với metric, càng không bảo đảm metric nhất quán với mục tiêu thực tế. Khi khe hở đó đủ rộng, mô hình sẽ khai thác nó — đây là *Goodhart's law* phiên bản học máy: khi một phép đo trở thành mục tiêu, nó thôi là một phép đo tốt. Điều đáng sợ không phải mô hình thất bại, mà là nó *thành công* theo cách bạn không tưởng tượng nổi: một mô hình phân vùng đạt Dice cao nhờ phình mặt nạ ra vì bạn đặt Tversky phạt âm tính giả nặng gấp bốn; một mô hình phát hiện đạt mAP cao nhờ đổ ra hàng trăm hộp điểm thấp mà metric không phạt.

Ba trường hợp hỏng cụ thể đáng thuộc:

- **Loss giảm mà metric không lên** — hoặc hàm thay thế không nhất quán, hoặc bạn đang đo trên một phân bố khác với phân bố huấn luyện (chương 8). Hai nguyên nhân này cần hai hành động ngược nhau, nên phải phân biệt trước khi sửa.
- **Loss vượt rồi đột ngột thành NaN** — hầu như luôn là một thành phần có mẫu số tiến về 0: Dice trên ảnh không có tiền cảnh, chia cho σ trong loss heteroscedastic, log của xác suất bằng 0. Phải truy nguyên từ bước *đầu tiên* xuất hiện chứ không phải từ chỗ nó nổ.
- **Loss đúng công thức nhưng sai giả định** — focal trên tập nhãn nhiều, MSE trên dữ liệu đuôi nặng, contrastive với mẫu âm toàn là dương giả. Đây là loại lỗi không bao giờ lộ ra trong đường cong huấn luyện.

CẠM BẮY

Trước khi tin bất kỳ đường cong loss nào, hãy kiểm tra giá trị tại lúc khởi tạo. Với C lớp và mô hình chưa học gì, cross-entropy kỳ vọng phải bằng $\ln C$: $\ln 10 \approx 2,30$ cho CIFAR-10, $\ln 1000 \approx 6,91$ cho ImageNet. Nếu epoch 0 in ra 0,4 thì bạn đang rò rỉ nhãn hoặc đánh giá nhầm tập; nếu in ra 15 thì logit đang nổ hoặc chỉ số nhãn lệch. Phép kiểm tra này mất ba mươi giây và bắt được nhiều lỗi hơn bất kỳ thứ gì khác cùng chi phí. Kèm theo nó là hai thói quen: tắt hết chính quy hoá để nhìn riêng *data loss*, và log từng thành phần riêng thay vì chỉ log tổng.

CẦN NHỚ

Loss không phải là công thức mà là một *phát biểu*: về phân phối của nhiễu, về việc mẫu nào đáng được chú ý, và về việc loại sai lầm nào đắt hơn. Nếu bạn không nói được phát biểu đó thành một câu tiếng Việt thì bạn chưa chọn loss — bạn mới sao chép nó. Và ở chỗ nào loss phẳng, ở chỗ đó mô hình mù. [T2]

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Hàm thay thế	Tiếng Anh là <i>surrogate loss</i> : hàm khả vi ta thực sự tối ưu, đứng thay cho metric vốn không khả vi. Nó chỉ hợp lệ khi nghiệm tối ưu của nó trùng nghiệm tối ưu của mục tiêu gốc; điều đó gần như không bao giờ được chứng minh trong thị giác.
••• Vì sao không tối ưu thẳng metric như mAP?	Vì metric là hàm bậc thang theo tham số: đổi trọng số một lượng vô cùng nhỏ thì nó hoặc đứng yên hoặc nhảy một bậc. Gradient của nó bằng 0 gần như mọi nơi nên vô dụng cho việc học.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
••• Cực đại hoá độ hợp lý	Chọn tham số làm dữ liệu đã quan sát trở nên khả dĩ nhất. Nó sinh ra hầu hết công thức loss: nhiều Gaussian cho MSE, Laplace cho MAE, Categorical cho entropy chéo.
•• Hàm Huber	Bậc hai trong bán kính δ , tuyến tính ngoài bán kính đó, nên một ngoại lai không kéo được cả mô hình. Giả định là ngoại lai hiếm và nằm xa; đặt δ sai thì bạn được cái tệ nhất của cả MSE lẫn MAE.
•• Heteroscedastic loss	Mạng xuất ra cả giá trị dự đoán lẫn phương sai của chính nó, và phương sai đó vào thẳng mẫu số của số hạng phạt. Kết quả là mô hình học luôn độ bất định của mình, dùng được cho khâu hợp nhất cảm biến ở hạ nguồn.
•• Vì sao hinge và cross-entropy hành xử khác nhau?	Hinge tắt gradient ngay khi mẫu vượt lề, nên mẫu đã đúng biến mất khỏi tầm nhìn. Cross-entropy không bao giờ tắt, nên nó cho xác suất hiệu chỉnh tốt hơn nhưng bị nhãn sai kéo vô hạn.
••• Focal loss	Nhân entropy chéo với $(1 - p_t)^\gamma$ để dim đóng góp của mẫu dễ. Ở $p_t = 0,9$ với $\gamma = 2$, hệ số là 0,01, tức chia đóng góp cho một trăm lần. Nó hỏng khi nhãn nhiều, vì nhãn sai trông y hệt mẫu khó.
•• Vì sao focal loss không chứa được tập đuôi dài?	Vì focal cân theo <i>độ khó của từng mẫu</i> , còn đuôi dài là vấn đề <i>tần suất của từng lớp</i> . Nếu mọi lớp đều dễ về thị giác thì focal không đổi gì; phải dùng loss cân theo số mẫu hiệu dụng.
•• Label smoothing	Thay nhãn one-hot bằng $(1 - \epsilon)$ cho lớp đúng và chia đều ϵ cho phần còn lại. Nó đặt trần hữu hạn cho logit đúng nên hiệu chỉnh xác suất tốt lên, nhưng làm các lớp co cụm lại nên hại cho truy hồi và chưng cất.
• Reverse KL	Chỉ phạt khi mô hình đặt khối lượng vào chỗ dữ liệu không có, nên nghiệm tối ưu bám lấy một mode. Hệ quả: đầu ra sắc nét mà thiếu đa dạng, ngược hẳn với forward KL vốn phủ hết mode và cho ảnh mờ.
••• Hệ số Dice	Hai lần phần giao chia cho tổng diện tích hai mặt nạ. Nó không cộng điểm cho pixel nền đúng, nên đoán “toàn nền” cho 99% độ chính xác pixel nhưng Dice bằng 0. Mẫu số tiến về 0 khi ảnh không có tiền cảnh.
••• GIoU	IoU trừ đi tỉ lệ phần rỗng của hộp bao nhỏ nhất chứa cả hai hộp. Nó vá đúng chỗ IoU bị phẳng: với hai hộp rời nhau, IoU đều bằng 0 còn GIoU cho $-0,78$ và $-0,94$, nên gradient lại chỉ đúng hướng.
•• Ghép cặp Hungary	Ghép một-một tối ưu giữa dự đoán và nhãn <i>trước</i> khi tính loss, khiến loss bất biến với thứ tự. Đây là thứ cho phép DETR bỏ hẳn bước triết phi cực đại.
•• InfoNCE	Loss đối chiếu một mẫu dương với nhiều mẫu âm trong một không gian nhúng. Hai thứ quyết định thành bại là nguồn mẫu âm và nhiệt độ; mẫu âm quá dễ thì loss về 0 mà không học được gì.
•• Vì sao loss theo pixel cho ảnh mờ?	Khi nhiều đáp án đều đúng, trung bình bình phương kéo đầu ra về <i>trung bình của các đáp án</i> , mà trung bình của nhiều ảnh sắc nét là một ảnh mờ. Mất mát tri giác và loss đối kháng ra đời để thoát khỏi chính hiện tượng này.
• Chưng cất tri thức	Huấn luyện theo phân phối mềm của một mô hình lớn thay vì theo nhãn cứng. Phân phối mềm chứa thông tin về sự tương tự giữa các lớp mà nhãn one-hot vứt bỏ hoàn toàn.
•• Vì sao trọng số của loss nhiều thành phần khó chỉnh?	Vì thang của từng số hạng phụ thuộc phân bố dữ liệu, nên bộ trọng số tinh chỉnh trên tập này thường vô nghĩa trên tập khác. Ngoài ra hai số hạng có thể xung đột gradient, và đó là bản chất của học đa nhiệm chứ không phải lỗi cài đặt.
••• Epoch 0 in ra loss 6,5 trên CIFAR-10 — nghĩa là gì?	Sai, vì giá trị kỳ vọng lúc khởi tạo phải là $\ln 10 \approx 2,30$. Cao hơn nhiều nghĩa là logit đang nổ hoặc chỉ số nhãn lệch; thấp bất thường thì ngược lại, nghi ngờ rò rỉ nhãn.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Loss giảm mượt rồi đột ngột thành NaN — nguyên nhân?	Gần như luôn là một thành phần có mẫu số tiến về 0: Dice trên ảnh không có tiền cảnh, chia cho σ trong loss heteroscedastic, hoặc log của xác suất bằng 0. Truy nguyên từ bước <i>đầu tiên</i> xuất hiện, không phải từ chỗ nó nổ.
●● Loss giảm đều mà mAP đứng yên — nguyên nhân?	Hoặc hàm thay thế không nhất quán với metric, hoặc bạn đang đo trên một phân bố khác với phân bố huấn luyện. Phân biệt bằng cách đo lại metric ngay trên chính tập huấn luyện: nếu ở đó metric cũng đứng yên thì lỗi nằm ở loss.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
mmdetection , detectron2	Focal loss cho phân loại anchor và họ GloU/DIoU/CIoU cho hồi quy hộp là mặc định của gần như mọi detector một giai đoạn	Hạ tầng, ít hết hạn; ý tưởng loss bền hơn tên kiến trúc
Dòng DETR (Deformable DETR, DINO-DETR)	Ghép cặp Hungary biến phát hiện vật thể thành bài toán dự đoán tập hợp và bỏ được triết phi cực đại	Phần được kế thừa rộng rãi là loss, không phải kiến trúc
nnU-Net	Tổ hợp entropy chéo cộng Dice là cấu hình mặc định, chính vì lý do mất cân bằng nền đã phân tích ở chương	Vẫn là đối chứng mạnh cho ảnh y sinh tính tới 9/2026
CLIP, DINOv2/DINOv3	Toàn bộ tín hiệu huấn luyện là contrastive: InfoNCE với nhiệt độ học được, không có nhân phân lớp nào	Nền của phần lớn mô hình nền tảng thị giác hiện nay
Bộ mã hoá tự động của các mô hình khuếch tán tiềm ẩn	Mất mát tri giác cộng một số hạng đối kháng là lý do ảnh giải mã không bị mờ như khi chỉ dùng MSE	Đây đúng là hiện tượng “trung bình của nhiều đáp án” đã nêu trong chương

MINI PROJECT SLAM · Hai hàm mất mát cho cùng một bài ghép cặp đặc trưng ORB · 1–2 ngày

Mục tiêu	kiểm chứng bằng số luận điểm trung tâm của chương — chọn loss là chọn hành vi — trên đúng bài toán bạn đang làm: quyết định hai mảnh ảnh quanh hai đặc trưng ORB có ứng với cùng một điểm 3D hay không.
Dữ liệu	một chuỗi EuRoC; sinh cặp dương và cặp âm từ chân trị pose cộng độ sâu stereo, hoặc đơn giản hơn là lấy các ghép cặp đã sống sót qua kiểm định hình học của chính ORB-SLAM3 làm cặp dương. Cắt mảnh 32×32 quanh mỗi đặc trưng; vài chục nghìn cặp là đủ.
Các bước	huấn luyện một CNN nhỏ theo hai cách, giữ nguyên backbone và mọi siêu tham số, chỉ đổi đầu ra và hàm mất mát: (1) đầu phân loại nhị phân với entropy chéo trên nhãn “cùng điểm / khác điểm”; (2) đầu nhúng với loss có biên — triplet hoặc InfoNCE. Sau đó tính khoảng cách trong không gian nhúng cho tập kiểm tra và vẽ hai biểu đồ phân bố, một cho cặp đúng và một cho cặp sai. Lặp lại phiên bản (2) với khai thác mẫu âm khó và vẽ lại.
Xong khi	bạn có ba con số cho mỗi cấu hình — tỉ lệ dương giả tại ngưỡng cho recall 95%, diện tích chồng lấn của hai phân bố, và khoảng cách giữa phân vị 5% của cặp sai với phân vị 95% của cặp đúng — và nói được loss nào tách tốt hơn ở <i>đuôi</i> , tức đúng vùng mà quyết định ghép cặp thật sự xảy ra.
Nổi với	C/18 cho vai trò của mẫu âm và nhiệt độ; B/10 vì kết quả phụ thuộc cả backbone chứ không riêng loss; B/11 nếu bạn định chạy bộ mô tả này thời gian thực.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì hai mô hình có thể cùng giá trị loss mà khác nhau ở độ hiệu chỉnh xác suất: hinge tất gradient ngay khi vượt lề nên điểm số của nó không có nghĩa xác suất, còn cross-entropy là proper scoring rule nên xác suất của nó đọc được. Quyết định có ngưỡng cần thứ hai, không cần thứ nhất.
- A2. Vì MSE là log-likelihood của một mô hình nhiễu Gaussian. Bạn không tránh được việc chọn giả định; bạn chỉ chọn nó một cách vô thức, và giả định đó sai ngay khi phần dư có đuôi nặng.
- A3. Vì entropy chéo theo pixel cộng điểm cho mọi pixel nên đoán đúng, mà nên chiếm 99%. Dice không cộng điểm cho nên nên nó bằng 0 tuyệt đối trong đúng tình huống đó — hai hàm đo hai thứ khác nhau trên cùng một dự đoán.
- A4. Vì IoU bằng 0 cho mọi cặp hộp rời nhau, bất kể xa gần: loss phẳng nên gradient bằng 0 và mô hình không nhận được tín hiệu nào về hướng phải đi. GIoU trừ thêm phần rỗng của hộp bao nhỏ nhất, cho $-0,78$ so với $-0,94$ trong ví dụ ở chương, nên hướng lại xuất hiện.
- A5. Label smoothing đặt một trần hữu hạn cho logit đúng nên chấm dứt cuộc đua logit vô hạn, và độ hiệu chỉnh xác suất tốt lên. Cái giá là các lớp bị co cụm trong không gian đặc trưng, nên chính đặc trưng đó dùng cho truy hồi hay chứng cất thì tệ đi.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn muốn một đứa trẻ dọn phòng cho sạch, nhưng “sạch” thì không đo được, nên bạn treo thưởng theo số món đồ được cất vào tủ. Đứa trẻ rất chăm và tuyệt đối theo chữ nghĩa: nó nhét cả rác vào tủ, còn những góc bạn quên tính điểm thì nó không đụng tới.

Máy học đúng như vậy. Thứ bạn thật sự muốn — robot không đâm vào ai — không đo được trong lúc dạy, còn máy thì chỉ hiểu một thứ duy nhất là một con số càng nhỏ càng tốt.

Mẹo thì đơn giản: đừng đi nhớ công thức, hãy trả lời một câu hỏi về dữ liệu của bạn — “sai số của tôi thường trông thế nào?”. Nếu sai số thường nhỏ và cân đối thì ra một công thức; nếu thỉnh thoảng có một mẫu hỏng nặng thì ra công thức khác. Mỗi câu trả lời là một công thức.

Cái giá gồm hai phần. Thứ nhất, máy tối ưu đúng con số bạn viết chứ không phải ý bạn định, nên chỗ nào bạn viết cầu thả thì nó thắng còn bạn thua. Thứ hai, chỗ nào con số không thay đổi thì máy hoàn toàn mù: hai cái hộp cùng đoán trượt, một cái sát và một cái rất xa, mà cùng bị chấm không điểm, thì máy không có cách nào biết cái nào tốt hơn.

Chỗ không rút gọn được: không ai chứng minh được rằng đẩy con số thay thế xuống thì thứ bạn thật sự muốn cũng tốt lên. Với hầu hết công thức đang dùng trong thị giác máy tính, đây là niềm tin dựa trên kinh nghiệm chứ không phải một định lý. Tôi không biết cách nói đơn giản nào che được chỗ hở này mà không nói dối.

Một câu: Máy làm đúng thứ bạn viết ra, nên phần khó không nằm ở chỗ dạy máy mà ở chỗ viết cho đúng.

1.9 Kết nối

Chương này là hệ quả trực tiếp của chương 4: loss là một trong bốn chỗ cài inductive bias, và mọi thứ vừa bàn chỉ là những cách khác nhau để phát biểu “tôi biết gì về lời giải”. Nó là *đối tác* của chương 10: kiến trúc cài giả định về *dạng* của hàm, loss cài giả định về *mục tiêu*; một giả định cài sai ở chỗ này thường phải trả giá ở chỗ kia. Và nó là *tiền đề ngược* của chương 7 và 8: nếu giao thức đánh giá sai thì việc loss có nhất quán với metric hay không cũng thành vô nghĩa.

Đọc tiếp theo bốn hướng. C/12-co-che-huan-luyen cho biết gradient của những loss này thực sự chảy thế nào và vì sao vài loss khó tối ưu dù đúng về nguyên lý. C/14-bai-toan-cv-loi là nơi focal, GIoU, Dice và Hungarian matching xuất hiện trong ngữ cảnh thật, kèm lý do lịch sử vì sao từng cái ra đời. C/17-mo-hinh-sinh phát triển đoạn về chiều của KL thành bốn triết lý khác nhau của chữ “giống dữ liệu thật” [17]. Cuối cùng, E/25-do-tin-cay-va-van-hanh nhặt lại sợi chỉ hiệu chỉnh xác suất mà label smoothing và *proper scoring rule* vừa thả ra [18].

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao “loss giảm nhưng mAP không tăng” là triệu chứng có *nhiều* nguyên nhân, và ba phép đo nào phân biệt được chúng?
2. Focal loss giả định gì về quan hệ giữa “mẫu khó” và “mẫu có nhãn sai”? Giả định đó bị vi phạm khi nào và hậu quả định lượng là bao nhiêu?
3. Cho hai hộp dự đoán cùng có IoU bằng 0 với hộp thật, vì sao GIoU phân biệt được chúng còn IoU thì không, và hai con số cụ thể là bao nhiêu?
4. Nếu bạn thay MSE bằng MAE mà kết quả tệ đi, điều đó nói gì về phân bố phần dư của bài toán?
5. Trong loss bốn thành phần, vì sao log giá trị từng thành phần là chưa đủ, và bạn cần log thêm cái gì?
6. Label smoothing cải thiện hiệu chỉnh xác suất nhưng làm hại điều gì, và vì sao điều đó quan trọng nếu bạn định dùng đặc trưng cho truy hồi?
7. Nêu một trường hợp trong hệ thống của bạn mà mục tiêu thực tế, metric và loss lệch nhau — và cái giá của khe hở đó là gì?

Nguồn

- [1] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- [2] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [3] A. Kendall, Y. Gal, and R. Cipolla, “Multi-Task Learning Using Uncertainty to Weigh Losses,” in *CVPR*, 2018.
- [4] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [5] T.-Y. Lin, P. Goyal, R. Girshick, et al., “Focal Loss for Dense Object Detection,” in *ICCV*, 2017.
- [6] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [7] F. Milletari, N. Navab, and S.-A. Ahmadi, “V-Net: Fully Convolutional Neural Networks for Volumetric Medical Image Segmentation,” in *3DV*, 2016.
- [8] H. Rezatofighi, N. Tsoi, J. Gwak, et al., “Generalized Intersection over Union,” in *CVPR*, 2019.
- [9] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [10] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [11] A. van den Oord, Y. Li, and O. Vinyals, “Representation Learning with Contrastive Predictive Coding,” in *arXiv:1807.03748*, 2018.
- [12] J. Johnson, A. Alahi, and L. Fei-Fei, “Perceptual Losses for Real-Time Style Transfer and Super-Resolution,” in *ECCV*, 2016.
- [13] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [14] L. A. Gatys, A. S. Ecker, and M. Bethge, “Image Style Transfer Using Convolutional Neural Networks,” in *CVPR*, 2016.
- [15] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.
- [16] S. Sagawa, P. W. Koh, T. B. Hashimoto, et al., “Distributionally Robust Neural Networks for Group Shifts,” in *ICLR*, 2020.
- [17] Y. Blau and T. Michaeli, “The Perception-Distortion Tradeoff,” in *CVPR*, 2018.
- [18] C. Guo, G. Pleiss, Y. Sun, et al., “On Calibration of Modern Neural Networks,” in *ICML*, 2017.